

HR Payroll Project

Tech Stack Learning Guide

How to understand, learn, and implement every package in your project

How to use this guide

Do not try to learn all of these at the same time. Each package has a section telling you exactly when to learn it, what resource to use, the best exercise to cement your understanding, and the one thing you must understand deeply before moving on.

Read each section when your project sprint demands it — not before, not after. Learning a package before you need it means forgetting it by the time you use it.

When	Package	Purpose
Right now	Flask, Werkzeug, python-dotenv	Web framework, password security, environment config
Sprint 2	Flask-SQLAlchemy, Flask-Marshmallow, Marshmallow-SQLAlchemy	Database ORM, serialization, validation
Sprint 3	Flask-Migrate	Database schema version control
Sprint 5	pytest, pytest-flask, ReportLab	Testing all routes, generating PDF payslips
Sprint 7	psycopg2-binary	PostgreSQL database driver

Flask

Learn in: Right now — Sprint 1

Flask is the web framework that everything in your project runs on top of. Your routes, blueprints, request handling, sessions, and application factory are all Flask. Before you can meaningfully use any other package in this list you need a solid understanding of how Flask thinks.

Docs	flask.palletsprojects.com
Quality	One of the best written docs in the Python ecosystem. Read directly — no course needed.
Start with	The Quickstart section, then the official Tutorial which builds a blog called Flaskr.

The best way to learn it

Follow the official Flaskr tutorial completely. Do not copy and paste — type every single line. The tutorial covers authentication, database, blueprints, and testing. Every single concept maps directly to something in your HR project.

What to pay attention to

- How `@app.route` works under the hood — not just how to use it
- Why `create_app()` is a function rather than module level code
- How the request context works and why it disappears after the route finishes
- What blueprints actually are beyond just a way to split files into folders

The exercise that cements it: After the tutorial, open your HR project and explain every Flask decision out loud without looking at notes. Why does each blueprint have a `url_prefix`? Why is `db` in `extensions.py` and not in `__init__.py`? When you can answer those without hesitation, Flask is yours.

Werkzeug

Learn in: Right now — Sprint 1

Flask is built on top of Werkzeug. You use it directly in your HR project for password hashing via `generate_password_hash` and `check_password_hash`. It also handles the development server and request/response utilities under the hood.

Docs	werkzeug.palletsprojects.com — read the Security Utilities section specifically
------	--

Focus on	generate_password_hash and check_password_hash only — that is all you need right now
Key concept	Hashing is a one-way operation. You cannot reverse a hash to get the original password back.

The exercise that cements it

Open a Python shell and run these commands one by one:

- Run `generate_password_hash('mypassword')` three times and notice the output is different each time even though the input is the same. This is the salt.
- Store one of those hashes in a variable and run `check_password_hash(hash, 'mypassword')` — it returns True.
- Run `check_password_hash(hash, 'wrongpassword')` — it returns False.

The one thing to understand deeply: The salt is a random value added to the password before hashing. This is why the same password produces a different hash every time. Without a salt, attackers could precompute hashes for common passwords and look yours up in a table. The salt makes that attack impossible.

python-dotenv

Learn in: Right now — Sprint 1

Reads your `.env` file and loads your secret key, database URL, and any other environment variables into your application automatically when it starts. Without this you would have to set environment variables manually in the terminal every single time you run the project.

Docs	github.com/theskumar/python-dotenv — the README covers everything in ten minutes
What it does	Loads <code>KEY=VALUE</code> pairs from a <code>.env</code> file into <code>os.environ</code> so your app can read them
Why it matters	Your <code>.env</code> file never gets committed to GitHub. Secrets stay on your machine only.

The exercise that cements it

- Create a `.env` file with `SECRET_KEY=mysecretkey` and `DATABASE_URL=sqlite:///test.db`
- Load it with `python-dotenv` and print both values using `os.getenv()`
- Try accessing the same variables without loading `python-dotenv` and see that they return `None`
- Add `.env` to your `.gitignore`, push to GitHub, and confirm the file is not visible in the repository

The one thing to understand deeply: Environment variables are not just a Flask concept. They are how every professional software system manages secrets across different environments. Development uses one database, production uses another. The code never changes — only the `.env` file. Understanding this now puts you ahead of most junior developers.

Flask-SQLAlchemy

Learn in: Sprint 2 — Employee Management

The ORM that lets you define your database tables as Python classes and query them using Python instead of raw SQL. Every model in your `models/` folder uses this. It is the bridge between your Python code and your database.

Docs	flask-sqlalchemy.palletsprojects.com first, then docs.sqlalchemy.org for deeper questions
Start with	Flask-SQLAlchemy docs — it is the simplified version built for Flask

What to pay attention to

- Column types and constraints — String, Integer, Boolean, DateTime, nullable, unique, default
- Relationships — `db.relationship()` and `db.ForeignKey()` — Employee links to User, Attendance links to Employee
- Querying — `filter_by()`, `filter()`, `.first()`, `.all()`, `.get_or_404()`
- The session — `db.session.add()`, `commit()`, `rollback()` — rollback is critical for payroll

The exercise that cements it

Write every query your HR project needs first as raw SQL, then write the same query using SQLAlchemy ORM syntax. Compare them side by side. This makes the ORM feel like a deliberate tool rather than magic you cannot see through.

Why rollback matters for your payroll engine: When your payroll calculation crashes halfway through processing 10 employees, you need the database to undo all partial changes automatically. Without understanding `db.session.rollback()`, you end up with corrupted payroll data where 5 employees are paid correctly and 5 are not.

Flask-Marshmallow + Marshmallow-SQLAlchemy

Learn in: Sprint 2

Handles two jobs in your project. Serialization — converting SQLAlchemy model objects to JSON to send to the client. Validation — checking that incoming JSON data from API requests is correct before touching the database.

Docs	marshmallow.readthedocs.io — start with core Marshmallow, not Flask-Marshmallow
Why core first	Flask-Marshmallow is a thin wrapper. Understanding core Marshmallow means understanding what is actually happening.
Key classes	Schema, SQLAlchemyAutoSchema, fields, validate, dump_only, load_only

Practice each job separately

- Serialization first — take a SQLAlchemy object and dump it to a Python dictionary using a schema. Confirm every field appears correctly.
- Validation second — take a raw dictionary with missing or wrong fields and load it through a schema. Read every validation error that comes back.

The exercise that cements it

Deliberately send bad data to one of your API routes and read what Marshmallow returns:

- Send a POST request with the title field missing from your employee creation route
- Send an invalid email address format
- Send a string where the salary field expects a number

Understanding exactly what Marshmallow catches and what it lets through is how you learn to write schemas that actually protect your API.

The one thing to understand deeply: `dump_only` fields appear in responses but are ignored if sent in a request. `load_only` fields are accepted in requests but never appear in responses. Your `id`, `created_at`, and `updated_at` fields should always be `dump_only` — the client should never be able to set these values.

Flask-Migrate

Learn in: Sprint 3 — Attendance

Manages database schema changes over time. Every time you add a column, rename a field, or create a new table, Flask-Migrate generates a migration script that describes the change. This means you can recreate your exact database on any machine by running migrations in order.

Docs	github.com/miguelgrinberg/flask-migrate — the README covers everything completely
Three commands	<code>flask db init</code> (once), <code>flask db migrate</code> (every model change), <code>flask db upgrade</code> (apply changes)
Key insight	A migration file is just a Python script with <code>upgrade()</code> and <code>downgrade()</code> functions. No magic.

The exercise that cements it

- Add a new column to your Employee model — for example `phone_number`
- Run `flask db migrate -m 'add phone number to employee'` and open the generated file
- Read every line of that migration file before running `flask db upgrade`
- Run `flask db upgrade` and confirm the column appears in your database
- Run `flask db downgrade` and confirm the column disappears

The one thing to understand deeply: Never edit a migration file that has already been applied to a database. If you made a mistake, create a new migration to fix it. Editing applied migrations causes the migration history to go out of sync and breaks your ability to reliably recreate the database schema.

Generates PDF payslips and monthly HR reports. Your Sprint 5 payslip feature and Sprint 6 reports feature both depend on this. It gives you full control over the layout, fonts, tables, and styling of every PDF your system produces.

Docs	docs.reportlab.com — focus on the Platypus section, not the low-level Canvas API
Two APIs	Canvas API (low level, manual positioning) vs Platypus API (high level, automatic layout)
Use Platypus	SimpleDocTemplate, Paragraph, Table, and Spacer handle 90% of what your payslips need

What to pay attention to

- SimpleDocTemplate — the container that builds your PDF from a list of elements called a story
- Paragraph — for all text content. Accepts basic HTML-like tags for bold, italic, and color
- Table — for your payslip earnings and deductions grid. The most important element for your project
- TableStyle — controls borders, background colors, padding, and alignment for every cell

The exercise that cements it

Build a miniature payslip before building the real one. Hardcode one employee's salary data and generate a single page PDF with a table showing earnings and deductions. Get the layout right in isolation before connecting anything to your database.

The one thing to understand deeply: ReportLab builds PDFs from a list called a story. You append elements to the story list in order — a Paragraph, then a Spacer, then a Table — and ReportLab flows them onto pages automatically. Think of it like building a document by stacking blocks, not by manually placing things at specific coordinates.

pytest is the testing framework that lets you write and run tests for every route, model, and function in your project. pytest-flask is a plugin that gives you a test client for simulating HTTP requests without running a live server.

pytest docs	docs.pytest.org — read the Getting Started guide first
pytest-flask docs	pytest-flask.readthedocs.io — short, covers fixtures and test client in 20 minutes
Key insight	Start testing code you already understand. Tests for code you do not understand feel impossible.

What to pay attention to

- The Flask test client — simulates HTTP requests and returns responses you can check in tests
- `conftest.py` — where shared setup lives. One fixture creates a test database, another creates a test user
- Test database — a separate SQLite database that gets wiped between tests. Never test against real data
- Fixtures — reusable setup and teardown functions that pytest injects into your test functions automatically

The exercise that cements it

Write a test that does all of the following in sequence:

- Registers a new user via POST `/auth/register`
- Logs in via POST `/auth/login` and confirms a 200 response
- Accesses a protected route and confirms a 200 response
- Logs out via POST `/auth/logout`
- Tries the protected route again and confirms a 401 response

Why this single test is so powerful: It covers your register route, login route, session management, `login_required` decorator, and logout route all at once. If this test passes, Sprint 1 is working correctly end to end. If it fails, the failure message tells you exactly which step broke.

The driver that lets Python talk to PostgreSQL. Without this, SQLAlchemy cannot connect to a PostgreSQL database. The binary version includes everything it needs without requiring you to install PostgreSQL development headers separately on your machine.

Docs	psycopg.org/docs — but you rarely need to read these directly
How you use it	Indirectly through SQLAlchemy. You almost never call psycopg2 functions yourself.
The key thing	The connection string format in your DATABASE_URL environment variable

The connection string — understand every part

```
postgresql://username:password@localhost:5432/hr_payroll_db
```

postgresql:// — tells SQLAlchemy to use psycopg2 as the driver

username — your PostgreSQL user (often 'postgres' locally)

password — your PostgreSQL password

localhost — the host where PostgreSQL is running

5432 — the default PostgreSQL port

hr_payroll_db — the name of your database

The exercise that cements it

- In Sprint 7, change your DATABASE_URL in .env from sqlite:///hr_system.db to the PostgreSQL connection string
- Run your existing test suite without changing a single line of application code
- If all tests pass, psycopg2 is working correctly and doing its job invisibly

The one thing to understand deeply: psycopg2 is a driver, not an ORM. It does not know what your tables look like or what your queries mean. It only knows how to open a connection to PostgreSQL and send raw SQL over it. SQLAlchemy sits above it and handles everything else. Your code does not change when you switch databases — only the connection string changes.

Quick Reference — All Stacks at a Glance

Package	What it does in your project	Best resource	Learn when
Flask	Web framework — routes, blueprints, sessions, request handling	flask.palletsprojects.com	Right now
Werkzeug	Password hashing for your auth system	werkzeug.palletsprojects.com/security	Right now
python-dotenv	Loads .env secrets into the app	github.com/theskumar/python-dotenv	Right now
Flask-SQLAlchemy	ORM — Python models become database tables	flask-sqlalchemy.palletsprojects.com	Sprint 2
Flask-Marshmallow	Serialize models to JSON, validate incoming data	marshmallow.readthedocs.io	Sprint 2
Marshmallow-SQLAlchemy	Bridge between Marshmallow and SQLAlchemy models	marshmallow-sqlalchemy.readthedocs.io	Sprint 2
Flask-Migrate	Version control for your database schema	github.com/miguelgrinberg/flask-migrate	Sprint 3
ReportLab	Generate PDF paylips and HR reports	docs.reportlab.com	Sprint 5
pytest	Write and run tests for every route and function	docs.pytest.org	Sprint 5
pytest-flask	Test client for Flask — simulate HTTP requests	pytest-flask.readthedocs.io	Sprint 5
psycopg2-binary	Driver that connects SQLAlchemy to PostgreSQL	psycopg.org/docs	Sprint 7

The single most important reminder

Do not read documentation for packages you are not using yet. Read Flask right now because you are building Sprint 1 right now. Read ReportLab in Sprint 5 when you are about to generate your first payslip. Reading documentation before you need it is preparation. Reading it at the moment of need is learning. Only one of those sticks.